Oracle Integration with AI. Part II.

Generating integrations + some comments.

May 10th, 2025.

Introduction.

This post contains the second chapter of my testing of OIC with AI. We will play with a more advanced OIC integration, asking for more functionality.

We are starting to see promising features!

We will be building on top of the previous chapter “OIC with AI Primer.pdf”.

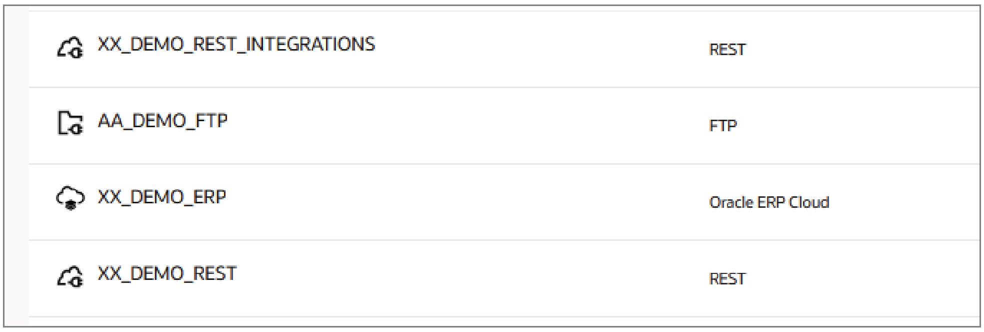
References:

- https://docs.oracle.com/en/cloud/paas/application-integration/integrations-user/use-ai-create-integration.html#GUID-0289F37A-6083-4617-A14C-0117F519F36D
- https://www.youtube.com/watch?v=Ah0_AMpRk2U
- https://www.youtube.com/watch?v=xT3sPDw-56w

Preparatory steps.

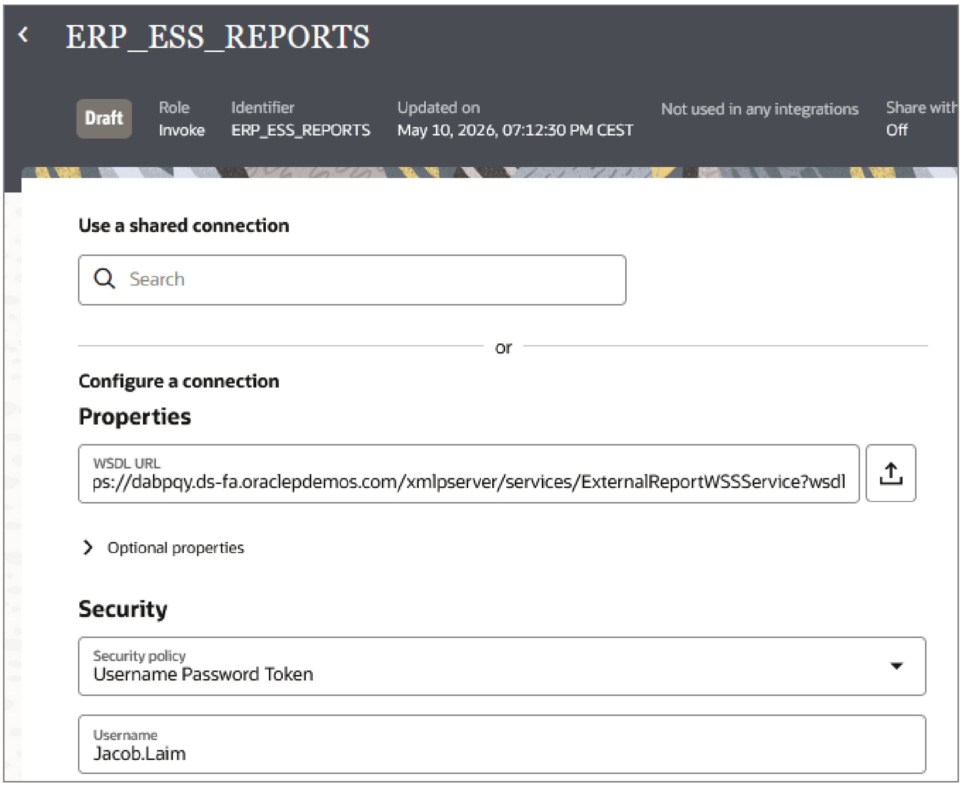
For this exercise you will need an OIC instance with AI enabled, step which can be done here if you have administrator privileges.

The connections are:

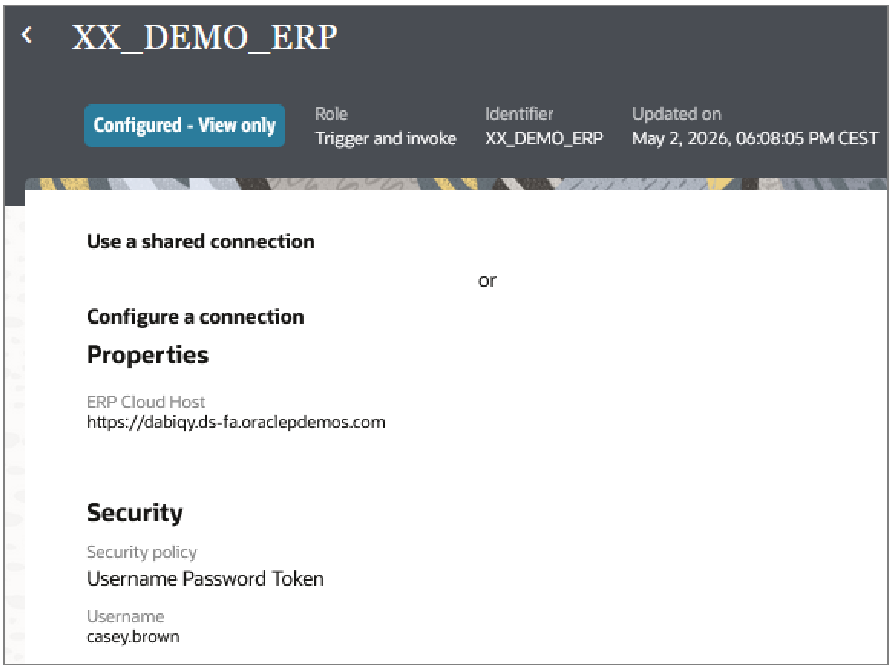


a) NEW. ERP_ESS_REPORTS.

https://dabpqy.ds-fa.oraclepdemos.com/xmlpserver/services/ExternalReportWSSService?wsdl

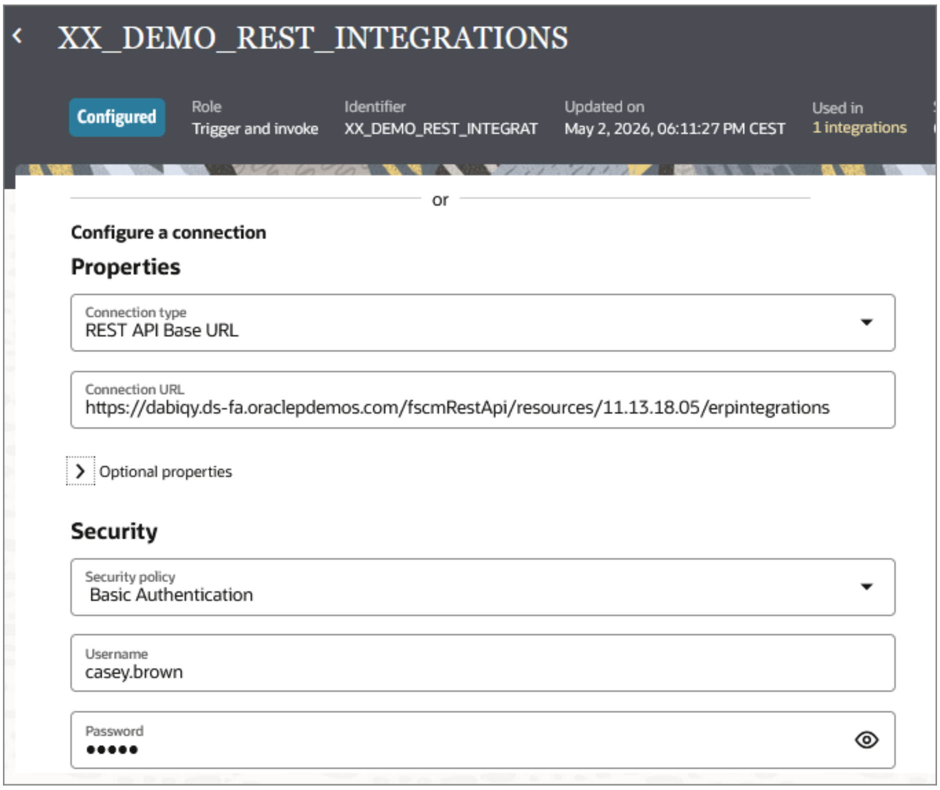


b) Erp connection.

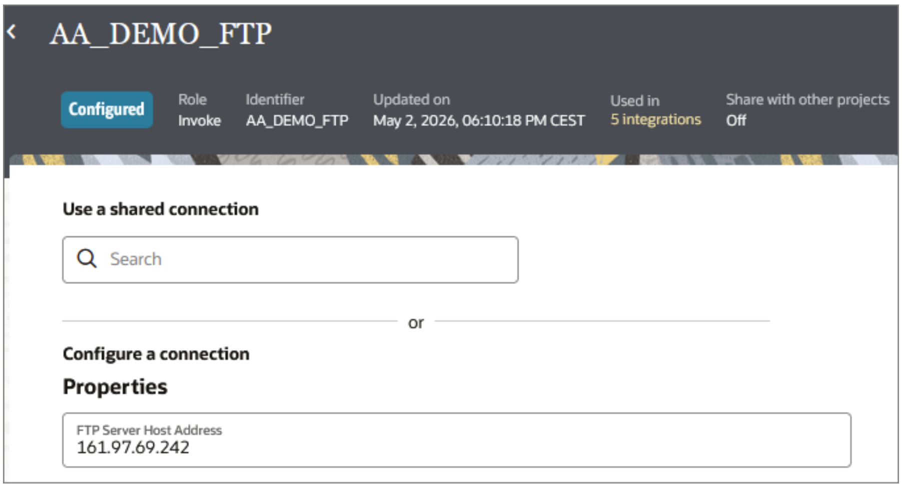


c) REST-Erpintegrations.

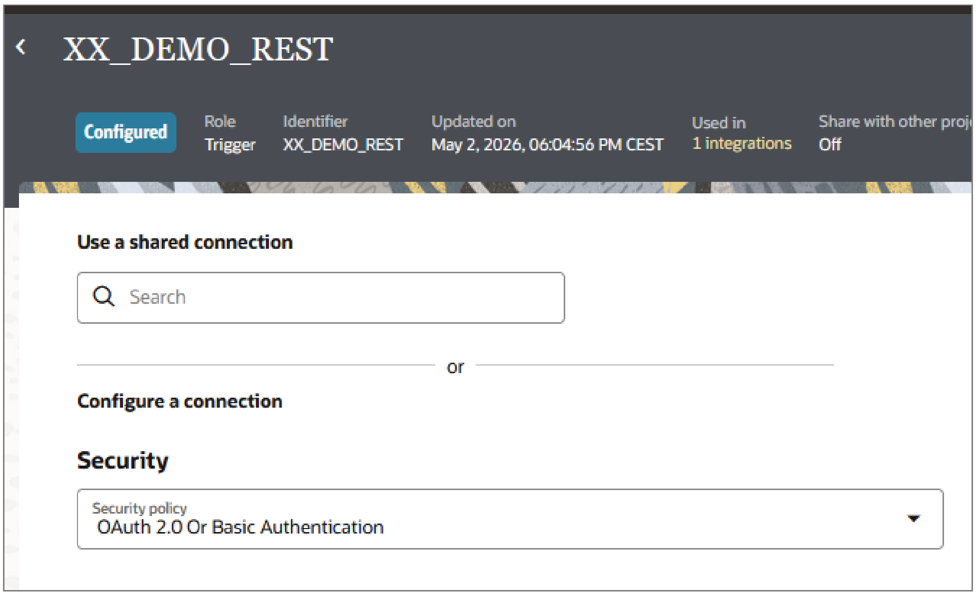
https://dabiqy.ds-fa.oraclepdemos.com/fscmRestApi/resources/11.13.18.05/erpintegrations



d) FTP connection.

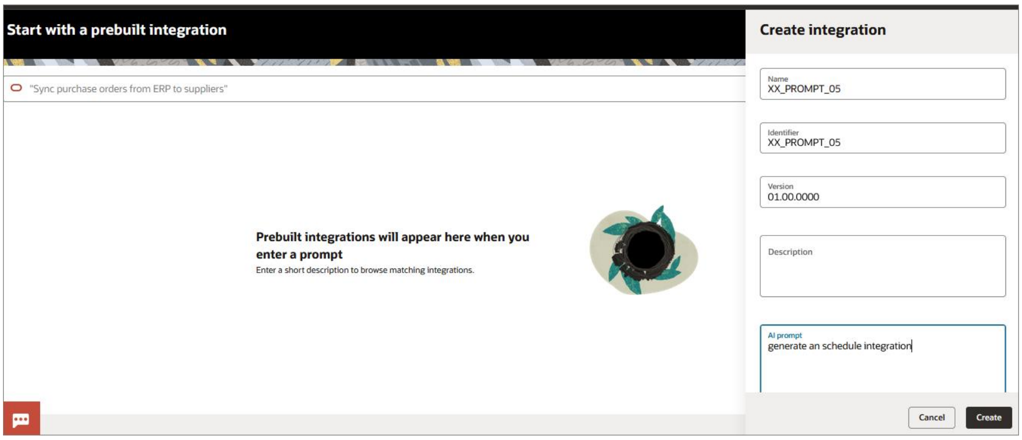


e) REST.

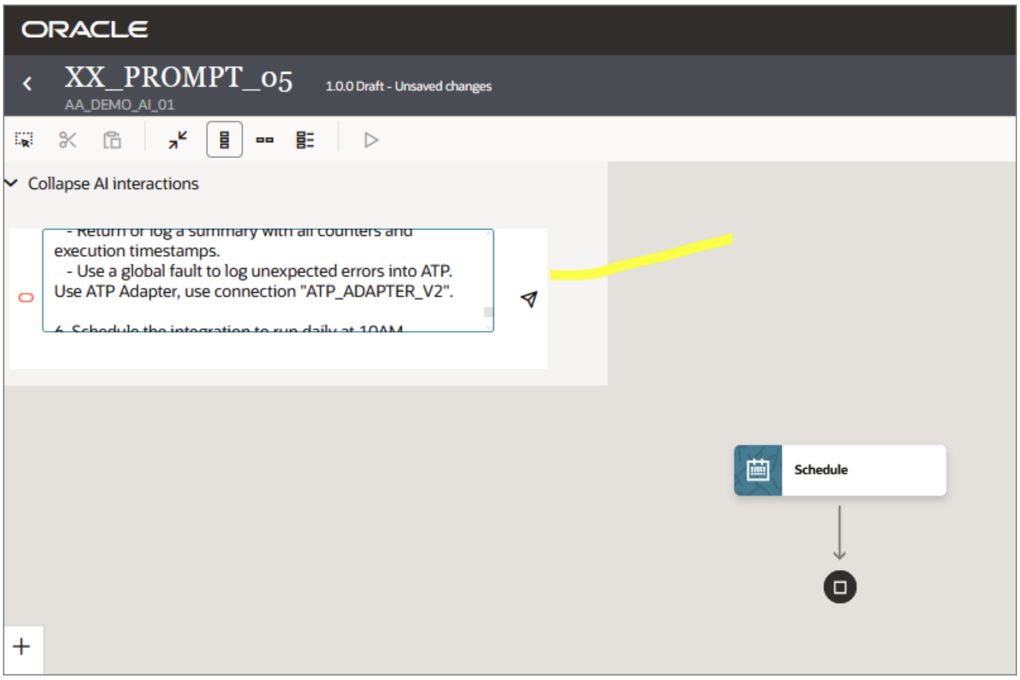


Our prompts.

1. I started with this step.



2. Then, I run my instruction.



3. The complete instruction to refine for this exercise would be:

NOTICE: I got this after quite a few iterations "trial and error".

Create one Oracle Integration Cloud (OIC) orchestration that performs the following end-to-end process.

Integration type:
- Scheduled orchestration (batch).

Steps:

0. Initialize 3 variables for counters.

1. Run a BI Publisher report in Oracle Fusion using the SOAP connection "ERP_ESS_REPORTS_SOAP".

- Report path: /Custom/Reports/Your_Report.xdo
- Output format: CSV only.
- Execute synchronously.
- Decode the Base64 response and store the CSV file in FTP server using connection "AA_DEMO_FTP".
- Node name: RUN_BIP.

2. Read the generated CSV file in FTP server using connection "AA_DEMO_FTP".
- Use Stage File Action with CSV format definition.
- The CSV contains multiple rows.
- Read the file into a repeating Row element.
- Node name: READ_CSV.

3. Loop through each CSV row (For Each).
For every row:
a) Call a REST API to get an authentication token.
- Use connection XX_DEMO_ERP_REST.
- Save token response.
- Increment token success or failure counter.
- Node name: GET_TOKEN.
- Maintain counter during execution for this node for success / failure

b) Check if previous node has been successful, then:

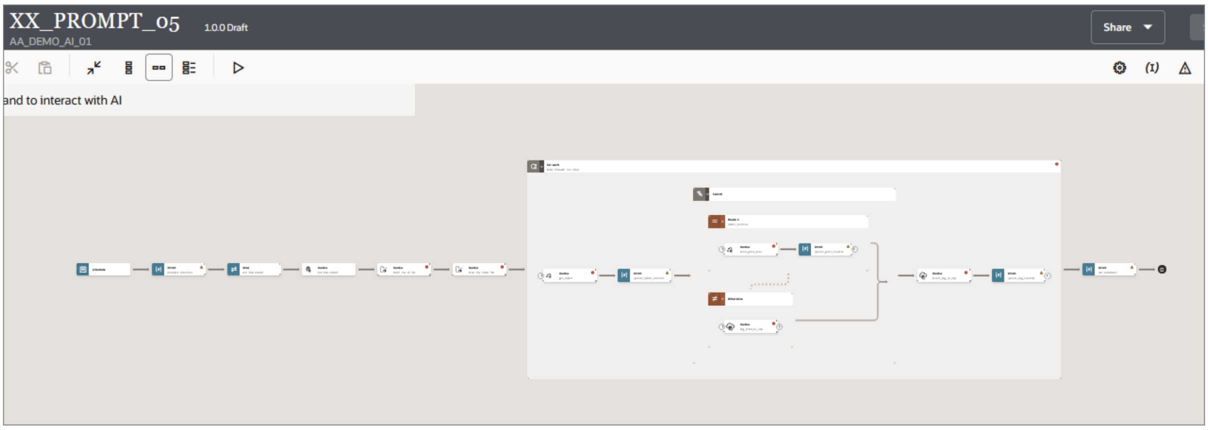
c) If previous step is successful, then call a second REST API using HTTP POST.
- Use connection XX_DEMO_ERP_REST.
- Use the token in the Authorization header.
- Map fields from the current CSV row into the request body.
- Increment POST success or failure counter.
- Node name: SEND_POST_DATA.
- Maintain counter during execution for this node for success / failure

Else: store log in ATP database using connection "ATP_ADAPTER_V2".

d) Insert a log record into an Autonomous Transaction Processing (ATP) database.
- Use ATP Adapter, use connection "ATP_ADAPTER_V2".
- Store run ID, row identifier, API status, error message (if any), and timestamp.
- Increment database insert success or failure counter.
- Node name: INSERT_LOG_ATP
- Maintain counter during execution for this node for success / failure

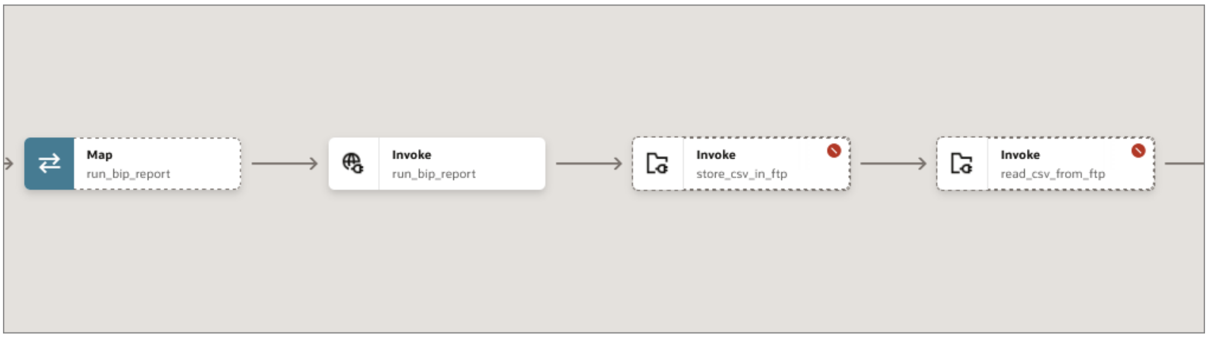
5. At the end of the integration:
- Return or log a summary with all counters and execution timestamps.

High level overview of the result.



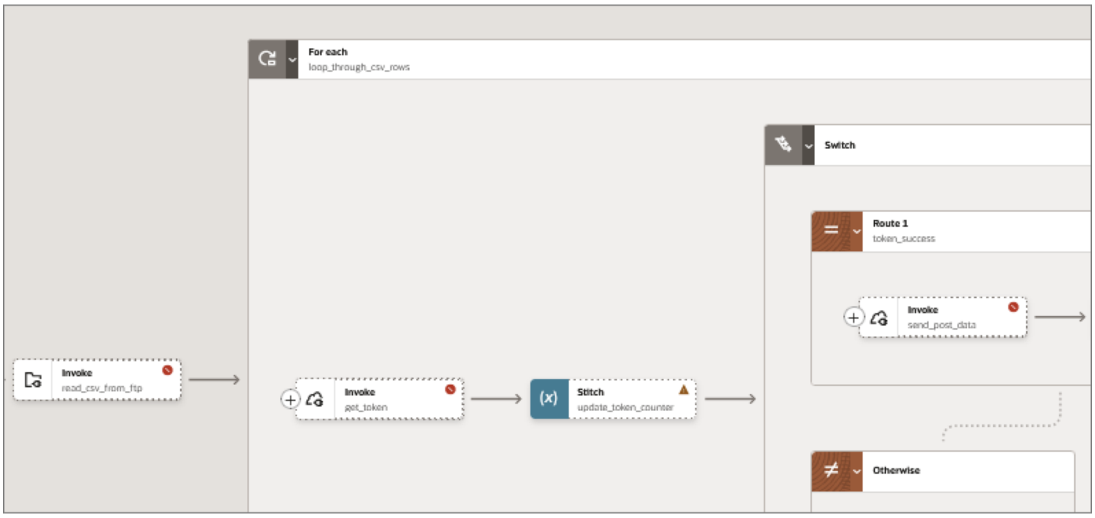
Notice we get the following:

1) Correct nodes with proper connections and proper names.

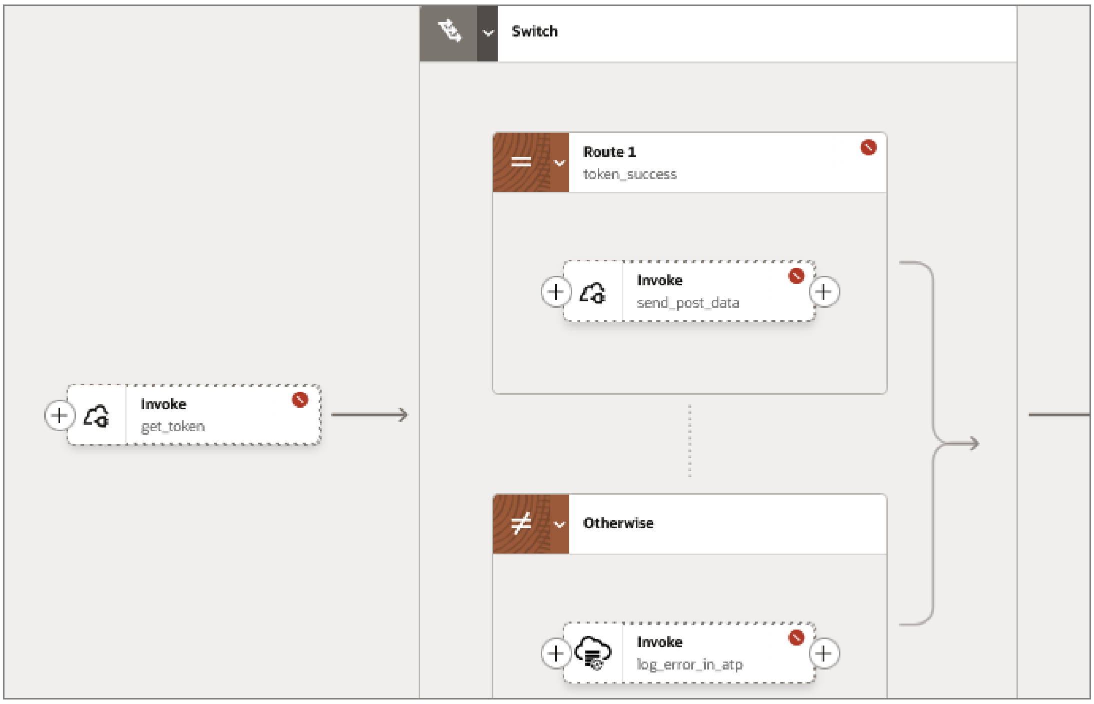


Even run_bi_report, shows completely setup and with map node. Good news!

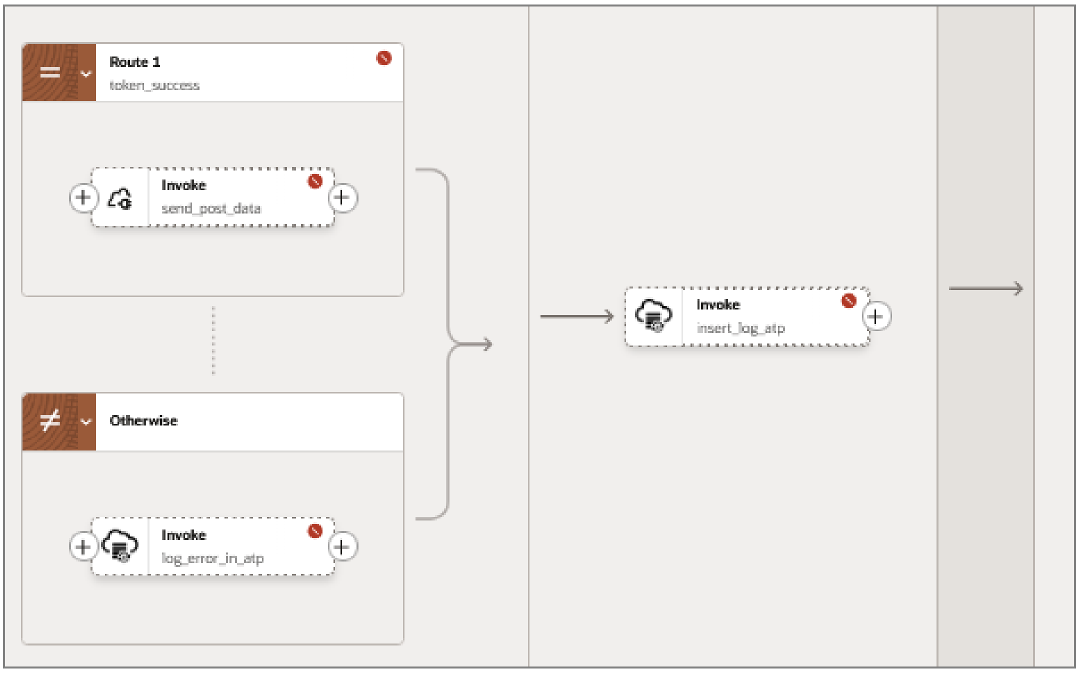
2) The Write activity to store the report (stage write is also another option).
3) The Read activity to read the report with format is already created.
4) The loop to read records is also created.



5) The question we asked to know if REST API call is ok, is also implemented.



6) We see also the ATP call to store the log records.



Conclusion.

Part 2 of Playing with OIC with AI.

We can generate a skeleton for a more advanced integration using a prompt using the new features in Oracle Integration with AI.

In the prompts we provide a lot of details, such as creating variables with names, paths to use when reading files, naming conventions, name of the files to store in archive, etc.

In this version, we can see the skeleton of the integration, but for now details provided seem not to be considered by the AI engines.

Notice that you have to manually complete a few steps, but is a good starting point, improving by days.

Code provided.

AA_DEMO_AI_02.car: project with integration demo and trials is provided.